

정소원

Sowon Jung

010-9349-1709 swj960515@gmail.com

github.com/ss-won velog.io/@ss-won

Frontend Developer

경력 5년

가천대학교 컴퓨터공학과 GPA 3.98 / 4.5

PROFILE

React·TypeScript 기반 5년차 프론트엔드 개발자입니다. 크립토 자산 지갑(Pockie), DeFi DApp(BiFi·BTCFi), 블록 탐색기(Bifrost Explorer), AI 데이터 분석 플랫폼(ThanoSQL)에서 제품 화면 구현, BFF 연동, 성능 개선, 테스트 자동화를 담당했습니다. 병목을 찾아 개선하는 과정을 좋아하고, 필요하면 BFF·FastAPI처럼 프론트엔드 바깥 영역도 직접 다룹니다. 기술적인 토론을 즐기고, 모르는 것을 편하게 물어볼 수 있는 분위기에서 가장 잘 일합니다.

EXPERIENCE

2024.09 - 현재

Frontend Developer

파이랩테크놀로지

Frontend Developer

Pockie 모바일 지갑/WebView 거래 제품, BTCFi 예치 상품, Bifrost Explorer의 제품 개발·운영 개선을 담당하고, 사내 Slack 기반 AI Agent(금쪽이)의 코드 리뷰 자동화 workflow를 개선했습니다. 주요 업무는 거래 화면 구현, 데이터 조회 구조 개선, 성능 최적화, 테스트 자동화, 품질 게이트 구성입니다.

TypeScript React Next.js TanStack Query Jotai NestJS
wagmi · viem Chakra UI Vitest Playwright Sentry

- Mobile App·Chrome Extension에서 동일한 BTCFi·Swap WebView 거래 화면을 재사용할 수 있도록 공통 UI 번들, postMessage 연동, BFF API 기반 거래 상태 조회 구조 구현
- 여러 RPC·외부 endpoint에 혼재돼 있던 사내 운영 DApp(BiFi, BTCFi, Swap, Gas Top-up)의 비즈니스 로직을 BFF 계약·TanStack Query hook 기반으로 통합해 WebView miniDApp별 거래 책임과 데이터 조회 경계 정리
- JPYC 스테이블코인의 예치·인출·수익 청구를 지원하는 BTCFi 파트너 제품 — 정책·수명주기가 다른 파트너 플로우를 하나의 제품 코어 위에서 운영하도록 구현하고, 비동기 입금·복구·epoch 출금·진행 대시보드를 초기 구축부터 릴리스까지 담당
- Blockscout 기반 Bifrost Explorer — 브릿지 필터, Next.js App Router 전환, 목록 화면 성능 개선, 테스트넷·메인넷별 Docker image 재사용을 위한 런타임 환경변수 주입 구조 전환
- 사내 Slack 기반 AI Agent(금쪽이) 코드 리뷰 workflow를 구성하고, 리뷰 근거·confidence verifier·Jira 티켓 검증·CI/테스트/타입/high severity 품질 게이트를 연결해 AI 리뷰 결과의 신뢰도와 운영 안전성을 개선
- UI Kit (디자인 시스템)에서 합성 컴포넌트(Tabs·Toast 등)의 동적 children 구조를 정적 매핑으로 표현할 수 없어 Figma Template V2 API를 도입 — 런타임에 인스턴스를 순회하고 연결된 자식 컴포넌트 템플릿을 재귀 실행해 실제 Figma 구성 기반 코드를 생성하는 파이프라인을 구현하고 Code Connect CLI와 연동해 Figma MCP 코드 생성 컨텍스트를 정형화

2021.07 - 2024.09

Full-stack Engineer

→ Frontend Developer

→ Frontend Part Leader

스마트마인드

Frontend Developer

고객 온프레미스 환경에서 ThanoSQL 분석 기능을 웹으로 사용할 수 있도록 JupyterLab Workspace, SQL Editor, Query Viewer를 개발했습니다. 프론트엔드 파트 리더로서 앱·공통 컴포넌트 중심의 pnpm/Turborepo 모노레포 구조를 구성했습니다.

TypeScript React Next.js Monaco Editor ANTLR pnpm · Turborepo
Playwright FastAPI Docker Compose

- ThanoSQL AI 데이터 분석 플랫폼 — 개발 도구가 초기화된 JupyterLab Workspace와 ThanoSQL 테이블 생성·조회용 SQL Editor를 개발하고, 정형·비정형 데이터 질의 흐름을 웹 제품 안에서 구성
- AI 팀이 정의한 ANTLR 문법을 Monaco Editor에 연결해 SQL 구문 강조·문법 오류 진단·키워드 자동완성을 제공하는 재사용 편집기 컴포넌트 개발
- 비정형 데이터(이미지·오디오·비디오) Query Viewer에 서버 페이지네이션·가상화·미디어 lazy loading 적용해 대용량 조회 화면 렌더링 부담 감소
- 앱 단위 라우트로 분산된 개발 구조를 pnpm·Turborepo 기반 모노레포와 공용 UI·유틸 패키지 구조로 정리 — 패키지 빌드 분리·빌드 캐시 활용·npm에서 pnpm 전환으로 평균 빌드 시간 **7분 → 1분** 단축
- 에너지 예측 정부과제(P2P-DC) — AI 팀이 파인튜닝한 PyTorch .pt 모델을 서빙하는 FastAPI 구현, Docker Compose 배포 환경 구성
- 프론트엔드 파트 리더 — 일정 관리, 업무 분배, 팀원 온보딩, 코드 리뷰

KEY ACHIEVEMENTS

01 /

프론트엔드 성능 병목 개선

Pockie(크립토 자산 지갑 서비스) 내 Gas Top-up·BiFi WebView와 Bifrost Explorer(Bifrost Network 블록 탐색기)에서 발생한 성능 병목을 서비스별 원인에 맞춰 개선했습니다. Pockie 계열은 데이터 조회 범위·중복 호출 구조를 정리했고, Explorer 계열은 초기 렌더링 범위와 WebSocket 이벤트 처리 방식을 조정했습니다.

Gas Top-up — Pockie 내 네트워크별 코인 충전 웹 서비스. 지원 토큰 범위와 서버 캐시 주기에 맞춰 자산·잔액 조회 범위를 줄이고, 불필요한 vault address·network asset·SDK 의존을 제거해 진입 시 호출 구조를 단순화

BiFi — Bifrost Network 예금·대출 DApp WebView. 토큰·네트워크 선택 시 네트워크별 bridge pair를 반복 조회하던 구조(1+N회)를 outbound/inbound 2회 캐시 조회로 전환해 초기 데이터 로딩 구조 단순화

Bifrost Explorer — Bifrost Network 블록 탐색기. /blocks·/txs·/tokens 목록 뷰에서 행마다 발생하던 시간 계산·상태 표시·차트·이미지 처리 비용을 줄이기 위해 초기 렌더링 범위 제한·단계적 렌더링·WebSocket 이벤트 30초 배치 처리 적용

Gas Top-up 호출 구조 단순화

보유 자산 수에 비례하던 자산·잔액 조회를 지원 토큰 기준 조회로 정리하고, vault address·network asset·SDK 중복 의존 제거

BiFi bridge pair 조회 구조 개선

네트워크별 반복 조회(1+N회)를 outbound/inbound **2회** 캐시 조회로 축소

Bifrost Explorer 목록 렌더링 최적화

최대 75행 조건에서 가상화의 mount/unmount 비용이 더 커 제거 — 초기 렌더링 범위 제한·단계적 렌더링·WebSocket 30초 배치 처리 적용

Route	LCP (before → after)	TBT (before → after)	개선
/blocks	3.7s → 1.9s	1,010ms → 270ms	LCP ↓49% · TBT ↓73%
/txs	3.7s → 2.2s	1,330ms → 200ms	LCP ↓41% · TBT ↓85%
/tokens	3.3s → 1.8s	510ms → 190ms	LCP ↓45% · TBT ↓63%

02 /

모바일 WebView 거래 제품 구현

서로 다른 호스트(Mobile App·Chrome Extension)와 파트너사 지갑(Hashport Wallet)에서 같은 거래 상품을 노출하기 위해, 화면·BFF·호스트 지갑·파트너사 지갑 사이의 책임 경계를 정리했습니다. 클라이언트 배포 없이 기능 노출 조건을 서버와 route 경계에서 제어할 수 있는 구조를 만들었습니다.

Pockie miniDApp ↔ BFF ↔ Mobile App / Chrome Extension — BiFi(Bifrost Network 예금·대출)·Swap·BTCFi 거래 화면이 BFF 계약을 기준으로 자산·거래 데이터를 소비하고, 지갑 서명·전송은 호스트 지갑(Mobile App 또는 Chrome Extension)에 위임. 같은 WebView 거래 화면을 여러 호스트에서 재사용하는 구조

BTCFi Partners ↔ Hashport Wallet — JPYC 스테이블코인 예치 상품을 파트너사 (Hashport Wallet) iOS·Android 모바일 지갑의 WebView에서 제공. 기존 dApp 웹 흐름을 모바일 지갑 안에서 유지하기 위해 postMessage 이벤트를 협의·설계하고, widget route로 WebView 접근과 일반 웹 접근을 분리해 상품 로직 재사용

BiFi·Swap·BTCFi WebView 거래 화면 구현

BiFi 예치·출금·대출·상환 4개 화면, Swap 견적 조회·승인·거래 데이터 생성·상태 조회를 Pockie BFF API와 연결

WebView 거래 책임 분리

거래 화면은 BFF 계약을 소비하고, 지갑 서명·전송은 Mobile App 또는 Chrome Extension의 호스트 지갑에 위임하는 구조로 정리 — 같은 WebView 거래 화면을 여러 호스트에서 재사용

플랫폼·버전별 출시 제어

iOS·Android·Extension 및 앱 버전별 feature flag와 하위 호환 fallback을 BFF config v2에 구현 — 클라이언트 배포 없이 기능 노출 서버 제어

Mobile App·Chrome Extension에서 동일한 BiFi·Swap·BTCFi 거래 화면 재사용 구조 구현
BTCFi Partners · Hashport Wallet WebView — 출시 3개월간 일본 액티브 유저 **750명대**, 실제 입금 **1억 엔대** 운영 중

03 /

사내 AI Agent 런타임 개선

Slack 기반으로 운영되는 사내 AI Agent(금쪽이)의 런타임 구조를 개선했습니다. 금쪽이는 코드 리뷰 자동화·Jira 연동·파일 읽기/쓰기 등 개발 workflow를 보조하는 에이전트로, 단순 자동화보다 사람이 최종 확인하는 운영 원칙 안에서 AI가 검토·요약·검증을 보조하도록 설계에 초점을 맞췄습니다.

금쪽이 — 사내 Slack 기반 AI Agent. Claude Agent SDK(→ Codex app server) 기반으로 MCP 도구를 활용해 코드 리뷰·Jira 티켓 검증·CI 결과 확인·파일 읽기/쓰기를 수행하며, Slack에서 요청을 받아 결과를 반환하는 구조로 운영

컨텍스트 주입 구조 개선

메모리·프롬프트 주입 구조를 core memory와 MCP on-demand search로 분리하고, 요청별 변동 정보를 프롬프트 tail로 이동해 정적 prefix가 안정적으로 유지되도록 개선. 메모리 주입 예산·우선순위 절단·관측 로그를 추가해 요청별 컨텍스트 주입량을 통제 가능한 구조로 정리

자동 리뷰 workflow와 품질 게이트 구성

Slack 기반 코드 리뷰 흐름에 리뷰 근거·confidence verifier를 추가하고, commit/PR에 Jira 티켓 형식이 포함된 경우 실제 이슈 정보를 읽어 추가 검증. CI·테스트·타입 오류·high severity 이슈가 없고 팀 기준을 충족한 경우에만 자동 승인 후보로 분류, 최종 결과물은 사람이 확인하는 운영 원칙 유지

MCP 도구 실행 구조 개선

Claude Agent SDK의 stdio 기반 MCP 중복 실행 구조를, 사용 가능한 tool 목록 조회 후 필요한 도구만 동적으로 선택·재사용하는 dynamic tools 구조로 전환. 병렬 호출 시 MCP 서버 중복 실행 부담 감소

Codex app server 전환과 read/write mode 통합

Claude Agent SDK 의존 런타임을 Codex app server 기반으로 전환하면서 read/write tool 권한을 별도 mode로 나누던 구조를 통합. 기존 classifier는 LLM 호출 비용·대기 시간이 추가되고 오탐 시 write mode로 이어지지 않는 한계가 있었음. mode 통합 후 write tool 사용 시 보안 effective gate와 감사 로그를 거치도록 재설계

프롬프트 인젝션·외부 코드 실행 방어와 감사 로그

write 작업에서 prompt injection·Slack 계정 탈취로 민감 도구가 악용될 수 있는 경로를 줄이기 위해 보안 gate 강화. 조직 repo 외부 코드를 임의로 clone·실행하지 않도록 제한하고, npm package 전역 업데이트처럼 환경을 오염시킬 수 있는 작업은 workspace sandbox로 격리. 차단 사유와 도구 호출 맥락은 감사 채널과 tool call journal에 기록

코드 리뷰 근거·Jira 티켓 검증·품질 게이트를 연결한 자동 리뷰 workflow 구성
요청별 컨텍스트 주입량과 MCP 도구 실행 범위를 관측·통제 가능한 구조로 정리
Codex app server 기반 전환과 effective gate 판단으로 런타임 비용·승인 UX·권한 모델 개선
prompt injection·외부 코드 실행·전역 package 변경 리스크를 workspace sandbox와 감사 로그로 통제

CERTIFICATIONS & ACTIVITIES

CERTIFICATIONS & ACTIVITIES

정보처리기사 2019
OSSCA — TS Handbook 한글화 (2020), githru-vscode-ext (2023)
SmileGate Membership AI 1기 (2021)
AUSG 1기 (2019)

LANGUAGES

영어 — 기초 비즈니스
일본어 — 기초 비즈니스 (JLPT N3)